

BrotherMode for Dummies

An illustrated guide for someone who has never heard the words agent, loop or hook, and is not yet convinced any of this is worth the trouble.

Written against BrotherMode **2.0.0-rc.4**, commit `471efdc` on branch `main`. Every command in this book was executed by the author against a throwaway project before the book was published, and every block marked *what it printed* is pasted output, not a reconstruction. Where a capability is unproven, the book says so in plain words instead of rounding up.

There are no productivity numbers in this book. Not one. BrotherMode has never been run through a real working day by anyone (`docs/KNOWN-LIMITS.md`), so a percentage here would be invented, and inventing one would be the exact failure this whole system exists to prevent.

Contents

1. Monday, and what actually goes wrong
2. Install it in ten minutes
3. Your first loop
4. The five questions, kept apart
5. How to correct it, and where the correction goes
6. Gates and deep dives: reviewing a machine's decision
7. The Documentation folder, toured
8. Working with a team
9. What to hand over, and what to keep
10. Other runtimes: what works, what is unverified
11. When it goes wrong
12. Glossary

CHAPTER ONE

Monday, and what actually goes wrong

Forget the product for a moment. Here is a Monday.

You run a small tea company. The order totals are wrong by a penny sometimes, and a customer noticed. You open your AI coding assistant and say: *add VAT to the order total, and please do not break the rounding again, we refunded someone last month over this.*

It does a good job. It writes the code, it explains the fix, you ship it. Good Monday.

Now here is what happens over the next three weeks, and every item in this list is a mechanical property of how these tools work, not a complaint about how clever they are.

Tuesday: the sentence is gone

You open a fresh conversation, because the old one got long and slow. You ask for a discount feature. The assistant writes `price * 0.9` using decimal numbers, because the sentence *we refunded someone last month over rounding* was in a conversation that no longer exists. You say it again. It apologises sincerely. It will not remember on Wednesday either.

That is not forgetfulness in the human sense. The assistant has no place to put that sentence. The conversation IS the memory, and the conversation ends.

Wednesday: two of them, one file

You get ambitious and run two assistants at once, one on VAT and one on discounts. Both decide the cleanest place to work is `pricing.py`. The second one saves over the first one's work. Nothing errors. You find out on Thursday when a test you were sure passed does not.

Thursday: it says it is done

You ask whether the VAT work is finished. It says yes, and it means it. It has no way to distinguish *I wrote code that should work* from *I ran the test and watched it pass*, because both feel the same from the inside of a sentence. You run the test yourself. It fails.

Friday: nobody can review anything

Your one actual engineer, Priya, asks why order totals are now recomputed at read time instead of stored. The answer is in a transcript. The transcript is four thousand lines long, it is on your laptop, and it is not organised by decision. She shrugs and rewrites it her way, which is a fourth different behaviour for the same field.

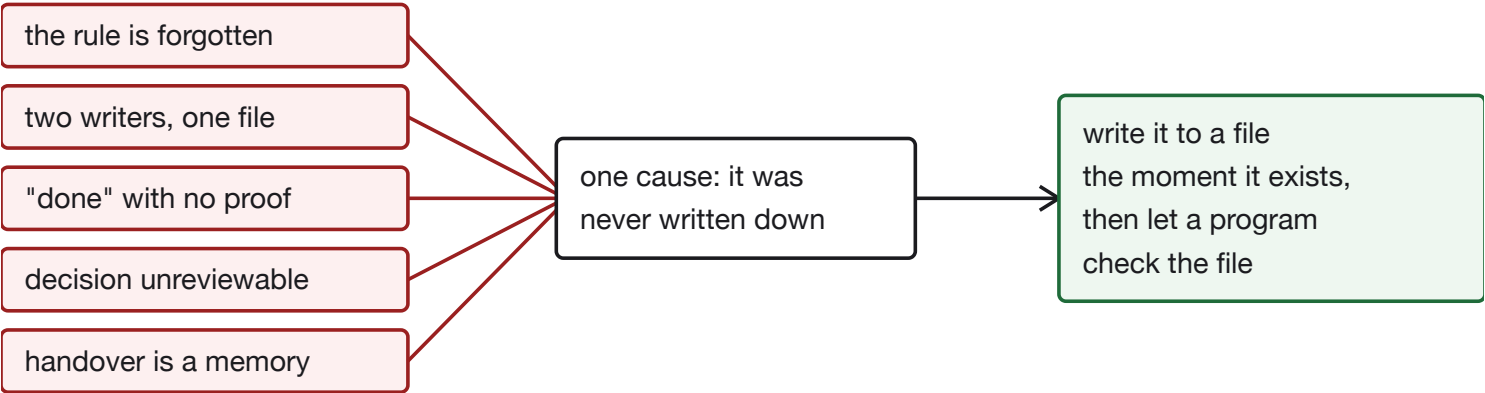
Three weeks later: the handover

You hire someone. They ask what the project is, what state it is in, and what decisions were already made and why. You have a repository and a memory. That is the whole handover pack.

What all five of those have in common

None of them is about the model being stupid. Every one of them is about something that **was never written down anywhere a machine could check it.**

The Monday to Friday list



The whole design in one picture. Models drift; disks do not.

What BrotherMode actually is

It is a small pile of Python scripts and one database file that sits in your project folder. It does five things, and each one is the direct answer to a day above.

THE DAY	WHAT BROTHERMODE DOES INSTEAD
Tuesday: the rule is gone	Your correction becomes a stored rule with an approval receipt. Before work starts, matching rules are looked up and the lookup is recorded.
Wednesday: two writers	One writer per file, enforced by a program that runs before every edit and refuses the second one.
Thursday: unproven "done"	Work cannot be marked complete without evidence. The command refuses.
Friday: unreviewable decision	One command writes a review pack with the live code, the callers, the tests, the rollback and a place for Priya's verdict. Her critical objection can refuse the decision outright.
The handover	One command writes a <code>Documentation/</code> folder, including a handover page written for a human who will never talk to an assistant.

Two properties are worth naming now, because they explain most of the design decisions you will meet later.

The programs, not the model, write the record. When a session ends, a hook (a small program the tool runs automatically at a fixed moment) writes what happened. The model does not get to narrate its own numbers. This is why the numbers can be trusted, and it is also why there are no numbers in this book: the honest ledger for real use is currently empty.

Refusal over guessing. Almost everything in this system prefers to stop and print a reason. You will see the word `refused` more in this book than the word `ok`. That is deliberate. A tool that guesses when it is unsure produces work you then have to audit; a tool that refuses produces one line you have to read.

Before you go further. BrotherMode is a release candidate. Its own limits page opens with the sentence "Everything here rests on tests, adversarial execution, and simulated lifecycles. No day of real founder work has yet been done through the V2 store." Its continuous integration has failed on Windows. If you want a finished product with case studies, this is not it. If you want a system whose honesty you can check, read on.

TRY THIS NOW

No install needed, and nothing here writes to your project. Open a terminal in any project you have and ask git the question the handover chapter will ask later:

```
git log --oneline -20
```

Now try to answer, from that output alone: which of those commits was a decision somebody made on purpose, and what was the reason? You will not be able to. That gap, between what the repository records and what you actually need to hand over, is the gap this book is about.

CHAPTER TWO

Install it in ten minutes

Two paths. The individual path is one command plus one check. The team path adds one decision about where the shared memory lives.

What you need first

- Python 3.9 or newer. Nothing else. No packages to install, no account, no network calls at runtime.
- Claude Code, if you want the automatic parts. Everything else works from a plain shell in any tool. Chapter ten is honest about which is which.

Step 1: get the files

```
git clone https://github.com/khalilmaaouni/BrotherModeUp.git \  
~/.claude/skills/brothermode
```

The path is the installation. Claude Code looks for skills under `~/.claude/skills/`, and the scripts work out their own location from there.

Step 2: prove it works on your machine before you wire it in

```
cd ~/.claude/skills/brothermode  
python3 tools/test_all.py
```

This runs every test suite, one at a time in its own process. It takes a minute or two. That is the cost of the isolation, not a hang. You are looking for one thing only, the last line: `ALL GREEN`. A skip or two is normal and is reported as a skip.

Do not memorise a test count. Counts change every time a test lands, and a book that pinned one would teach you to distrust the book instead of the tree.

Step 3: wire the five hooks

A hook is a small program your tool runs at a fixed moment, without asking the model to remember. BrotherMode has five, and the installer writes all five. Run the dry run first: it prints every change and writes nothing.

```
python3 ~/.claude/skills/brothermode/scripts/install.py --dry-run
```

What it printed, from a run into a sandboxed home directory so nothing real was touched:

```
[dry-run] BrotherMode 2.0.0-rc.4
[dry-run]   target:   /tmp/bmbook/fakehome/.claude/skills/brothermode
[dry-run]   settings: /tmp/bmbook/fakehome/.claude/settings.json
[dry-run] files: 168 file(s) would be copied
[dry-run] hooks: 0 BrotherMode entry(ies) replaced, 5 installed:
           SessionStart, SessionEnd, Stop, PreCompact, PreToolUse
[dry-run] hooks: every hook entry not owned by BrotherMode was left in place, in order.
[dry-run] smoke: skipped; nothing was written to test.
```

Happy with that? Drop the flag and run it for real.

```
python3 ~/.claude/skills/brothermode/scripts/install.py
```

Here is what each of the five actually buys you, because "five hooks installed" tells you nothing.

HOOK	FIRES WHEN	WHAT YOU LOSE WITHOUT IT
SessionStart	a session opens	the session begins with no idea what the standing rules are
SessionEnd	a session closes	nothing is recorded, so there is no ledger and no honest numbers ever
Stop	each assistant turn ends	the nudge when a day of work has no written log
PreCompact	just before the tool discards old context to free room	your uncommitted work, and the thread of what you were doing, at the exact moment the session is most likely to lose both
PreToolUse	before every file edit	the one writer per file guarantee. Without this the fence is a note, not a boundary.

Step 4: prove the fence is live, not just wired

This is the step people skip and should not. Wired and working are different things.

```
python3 ~/.claude/skills/brothermode/scripts/doctor.py
```

What it printed:

```
BrotherMode doctor: fence hook
settings: /tmp/bmbook/fakehome/.claude/settings.json
note: fence command: python3 ../tools/bm_fence_hook.py
OK: for each of Edit, Write, MultiEdit, NotebookEdit, the wired hook denied a
    foreign write and allowed the owner's own write, in a throwaway project
    that has been deleted.
```

What this does NOT prove, stated so it is not assumed:

- The fence FAILS OPEN by design. A missing store, a corrupt store, a store with no active claims, or any bug in the hook allows the write and prints a line starting 'bm_fence_hook: FAILING OPEN'. A hook that fails closed would brick editing, which is a worse failure than an unenforced fence, so this is a choice and not an oversight.
- Bash is NOT gated. Shell redirection, `sed -i`, `tee`, `python -c` and the rest write files without passing any hook.
- Claude Code loads settings at session start, so a settings file corrected during a session is live at the NEXT session, not this one.

Read the second half twice. Doctor built a throwaway project, claimed a file under one session, asked the hook you actually wired to approve an edit from a different session, watched it refuse, then asked again as the owner and watched it allow. Then it told you three things it did not prove. That paragraph is the most useful output in the whole install, and most tools would not print it.

The Bash hole is real and is policy, not a bug. The fence stands in front of the structured edit tools. It does not stand in front of `sed -i`, `tee`, a shell redirect, or `python -c`. If your team treats "run a shell command that rewrites the file" as normal, the fence will not save you. Chapter eight tells you how to work with that instead of pretending it away.

Step 5: point the memory somewhere

The vault is a plain folder of markdown and text files: session logs, the machine-written ledger, and optionally a model of how you like to work. Nothing in it talks to a server.

```
cp -R ~/.claude/skills/brothermode/vault-template ~/BrotherModeVault
export BROTHERMODE_VAULT="$HOME/BrotherModeVault"
```

Put that `export` line in your shell profile so it survives a restart.

Step 6: restart, then check

Hook configuration is read when the tool starts, so a session already running does not have your new hooks. Restart, then:


```
python3 ~/.claude/skills/brothermode/tools/bm_score.py
```

On a fresh install most checks say `NO-DATA` . That is the correct answer, not a failure. You have no history, and this tool refuses to invent one. A check without data says `NO-DATA` , never `PASS` .

The team path: two extra decisions

Where the project store lives. Each project gets a `.brothermode/` folder holding a small database. It is per project, not per person, and it lives next to the code. The usual choice for a team is to keep it out of git (add `/.brothermode` to `.git/info/exclude`) and let each person have their own, using the store for coordination inside one machine and the generated `Documentation/` folder, which IS committed, for coordination between people. Chapter eight walks through why.

Whose vault is whose. The vault holds your session logs and notes. Give each person their own. It is a folder; there is nothing clever to configure.

TRY THIS NOW

Do the whole thing into a sandbox, so you can look at the result before it touches your real configuration. This is exactly what the author did to produce the output above:

```
export HOME=/tmp/bmtry ; mkdir -p $HOME/.claude
python3 ~/.claude/skills/brothermode/scripts/install.py --dry-run
```

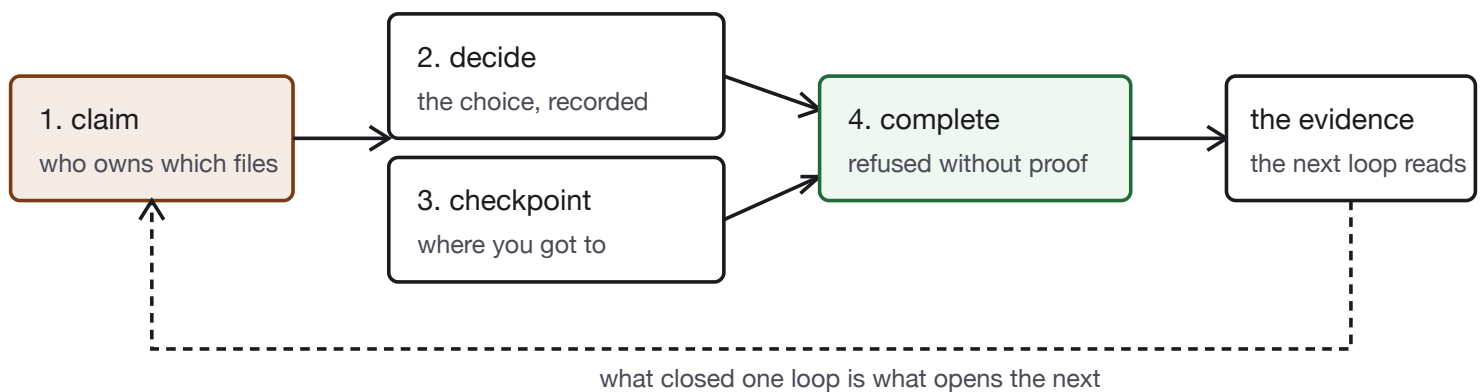
Read the list of five hooks and the "Still manual, on purpose" section at the bottom. Then open a fresh terminal, so you get your real `HOME` back, before you install for real.

CHAPTER THREE

Your first loop

A **loop** is one unit of work with a beginning, a middle and a provable end. That is the whole idea. It is called a loop rather than a task because the end feeds back: the evidence you produce at the end is what the next loop reads at its beginning.

Four commands. Everything below was run in a throwaway project called `tea`leaf : a git repository with one module, `src/pricing.py` , and one test.



Four commands. The only one that can refuse you is the last.

Command 1: claim the work and the files

Two ideas in one command. You are naming a unit of work, and you are declaring which files it is allowed to write. That second half is the fence.

```
python3 tools/bm_store.py claim vat-rounding \
  --lifetime persistent \
  --objective "Add VAT to the order total without floating point rounding drift" \
  --files src/pricing.py \
  --session "$ME" \
  --tier T1 \
  --check "python3 -m unittest discover -s tests -q"
```

```
claimed 'vat-rounding' as lifecycle f1a2f8b449194dd2bf608831cd440a1d
(version 1, session bm1-c03e7b279b1cfe4b835074f8)
```

Three of those flags deserve a sentence each.

--check

The command that will prove the work. You write it *before* you do the work, when you still have to think about what proof would look like. Afterwards you will accept anything.

--session

Who holds the fence. This is not your name; it is a label for one working session, produced by a helper (see the box below).

--tier

How much effort you are budgeting: T1 for a single focused session, T2 for a few parallel helpers, T3 for a fleet with divided scopes. It exists so that spend can be judged against a declared intent instead of a feeling.

The long hex string is a **lifecycle**: a permanent identity for this unit of work that nothing else will ever reuse. And `version 1` matters more than it looks. Every later command must tell the store which version it last saw. If the record moved in between, the command refuses instead of overwriting. You will see that refusal in chapter eleven, and you will be glad of it.

Where \$ME comes from. Do not invent a session label. Ask for it:

```
ME=$(python3 tools/bm_fence_hook.py session-label --session-id sess-mon-morning)
echo $ME
```

```
bm1-c03e7b279b1cfe4b835074f8
```

The label is derived from a secret token stored inside your project, so it is stable for that session and cannot be forged by another one. A label you typed by hand would mint a token nobody else can reproduce, and would quietly lock you out of your own records. Your labels will not match the ones printed in this book; that is correct.

Command 2: record the decision, not just the code

When you make a choice that a future person would ask about, spend one line on it.

```
python3 tools/bm_store.py decide f1a2f8b4... --version 1 \
  --topic "money representation" \
  --text "Keep every amount in integer pence and round VAT half-up at the order level, not per
line."
```

```
decision 1 recorded for f1a2f8b449194dd2bf608831cd440a1d (version 2)
```

Note that it hands you back the new version number. That is a small kindness with a history: an earlier version bumped the version silently, so your very next command failed against a number you had never been told.

Command 3: checkpoint, before you might die

A checkpoint says where you got to and what comes next. Write it before anything risky, not after.

```
python3 tools/bm_store.py checkpoint f1a2f8b4... --version 2 \  
  --next "write the failing test for a 5 percent VAT on 900 pence" \  
  --blockers "none"
```

```
checkpoint 1 recorded for f1a2f8b449194dd2bf608831cd440a1d (version 3)
```

This is the same instinct a database has when it writes its log before its data. If the session dies now, what survives is not just the files but the *next step*, which is the part that actually hurts to lose.

Command 4: complete, and try to cheat

Here is the interesting one. Try to close the work without proof:

```
python3 tools/bm_store.py complete f1a2f8b4... --version 3
```

```
refused (missing-evidence): active -> complete requires non-empty evidence  
(the check_cmd result)
```

There is no flag to skip that. So run the check you declared at claim time:

```
python3 -m unittest discover -s tests -q
```

```
Ran 1 test in 0.000s
```

```
OK
```

Then close it, pasting what you actually saw:

```
python3 tools/bm_store.py complete f1a2f8b4... --version 3 \  
  --session "$ME" \  
  --evidence "python3 -m unittest discover -s tests -q => Ran 1 test, OK"
```

```
complete: 'vat-rounding' (lifecycle f1a2f8b4...) is now complete at version 4
```

Nothing stops you writing a lie in that string. But a lie now has to be typed deliberately, into a field whose only purpose is proof, and it will sit in the generated handover document under the heading "the command that proves it". That is a very different act from saying "done" in a chat.

Where all of this shows up

Ask the store to render itself:

```
python3 tools/bm_store.py dashboard
```

```
<!-- BEGIN GENERATED BROTHERMODE STATE (edit outside these markers only) -->
_Generated by bm_store.py, 2026-07-30T01:49:17Z._

## active
- vat-rounding (f1a2f8b449194dd2bf608831cd440a1d, version 1, persistent) [T1]
  owner-session: bm1-c03e7b279b1cfe4b835074f8
  objective: Add VAT to the order total without floating point rounding drift
  files: src/pricing.py

<!-- END GENERATED BROTHERMODE STATE -->
```

That text is also written into a `STATE.md` file in your project. Look carefully at the markers: everything between them is regenerated, and anything you write *outside* them is left alone forever. That rule (generated output never destroys human text) runs through the whole system and has tests behind it.

TRY THIS NOW

Build a throwaway project and run the whole loop. It takes two minutes and touches nothing you care about.

```
mkdir -p /tmp/firstloop && cd /tmp/firstloop
git init -q .
echo "/.brothermode" >> .git/info/exclude
echo "print('hello')" > app.py
git add -A && git commit -qm start

BM=~/.claude/skills/brothermode
python3 $BM/tools/bm_store.py init
ME=$(python3 $BM/tools/bm_fence_hook.py session-label --session-id try-1)
python3 $BM/tools/bm_store.py claim greeting --lifetime ephemeral \
  --objective "make it greet by name" --files app.py --session "$ME" --tier T1 \
  --check "python3 app.py"
```

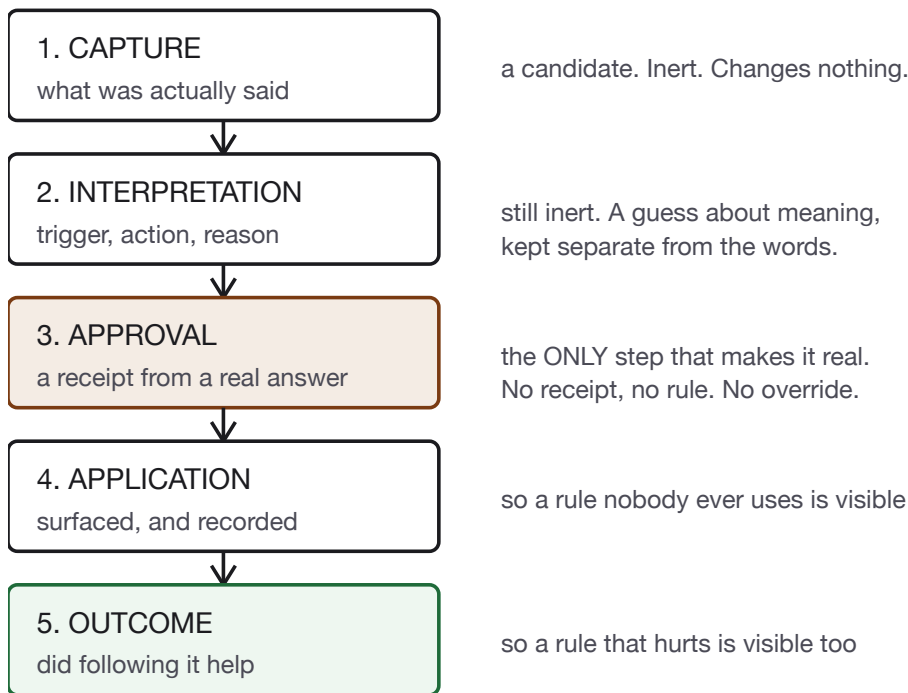
Copy the lifecycle it prints, then try to complete it with no evidence and watch it refuse. That refusal is the moment the system stops being an idea.

CHAPTER FOUR

The five questions, kept apart

Most tools that claim to learn from you collapse five different questions into one. Then you cannot tell a good idea from a bad habit, because the record of both looks the same.

Here they are, separated:



Five questions, five records. Collapse any two and you lose the ability to answer either.

Why the separation earns its keep

Suppose a rule turns out to be wrong. With the five kept apart you can ask which one broke:

- **Capture** was wrong: you never said that. The raw text settles it.
- **Interpretation** was wrong: you said it, but the rule written down means something wider than you meant.
- **Approval** was wrong: the rule was right for that moment and should not have been made standing.
- **Application** was wrong: the rule is fine, it was retrieved for work it does not apply to.
- **Outcome** was wrong: everything worked as designed and the rule is simply a bad rule.

Those are five different fixes. A single "learned preferences" list cannot tell you which one you have.

Stage 1 and 2: capture

Capturing is one command, and it changes nothing:

```
python3 tools/bm_learn.py capture \  
  --trigger "when I touch money in this project" \  
  --action "keep every amount in integer pence and never use a float" \  
  --because "a float rounded a customer invoice wrong last month" \  
  --scope project
```

```
captured fd93a141 (pending, nothing changes until you approve it)  
heads up, this looks like more than one rule: contains 'and never', which  
usually joins two rules  
approval will refuse it unless you split it or pass --override-reason
```

It caught something real. "Keep amounts in pence" and "never use a float" are two rules, and a rule that is secretly two rules cannot be graded, because you never know which half was followed. Split it:

```
python3 tools/bm_learn.py capture \  
  --trigger "when I touch money in this project" \  
  --action "keep every amount in integer pence" \  
  --because "a float rounded a customer invoice wrong last month" \  
  --scope project
```

```
captured 9b832875 (pending, nothing changes until you approve it)
```

`--scope` is how wide the rule reaches: `global`, `project`, `domain`, `artifact`, `relationship` or `tool`. A preference about this codebase is not a law about all your work, and the scope is where you say so.

Stage 3: approval, in two halves

This is the part of the design worth understanding, because it is unusual. Approving a rule takes two commands, not one.

```
python3 tools/bm_learn.py grant-approval 9b832875 \  
  --answer "yes, integer pence everywhere, that bug cost us a refund"
```



```
approval receipt b4440dc0 for candidate 9b832875
  expires 2026-07-30T02:05:50Z (900 seconds), one candidate, one use
  it dies if the candidate or the rule text changes before you approve
```

```
RECEIPT TOKEN, shown once and stored nowhere:
a913dee38dd443a97726095e1c92875492d9f400263dc1ac
```

```
spend it: bm_learn.py approve 9b832875 --receipt <token>
or put it in BM_APPROVAL_RECEIPT so it stays out of your shell history
do not paste it into a log, a transcript or a commit message
```

```
python3 tools/bm_learn.py approve 9b832875 \
  --receipt a913dee38dd443a97726095e1c92875492d9f400263dc1ac \
  --ref "founder said yes in the Monday session; the refund is in the ledger"
```

```
approved as rule 4873a92d
  4873a92d [project:tealeaf, approved] v1
    When: when I touch money in this project
    Do : keep every amount in integer pence
```

Why the token? Because before it existed, any process that could run the tool could manufacture an approved rule, and the reference it recorded said "run by the founder" whether or not anyone was there. The token is minted only when someone records what the human actually answered, it is bound to that exact rule text, it expires in fifteen minutes, and it works once. Change a word of the rule after the question was asked and the token dies. There is no override and no break-glass.

Try to spend it twice:

```
refused (not-pending): candidate 9b832875 is 'approved', only a pending candidate
  can be approved
```

What the receipt does not claim. It does not prove *which* human answered. It proves an answer was given about this exact thing and has not been spent. No wording anywhere in the product says otherwise, and that restraint is the reason the mechanism is worth anything.

Stage 4: application, and the honest surprise

Before substantial work, ask what rules apply. This both retrieves and records the retrieval:

```
python3 tools/bm_learn.py apply \
  --query "add VAT to the order total" --session sess-mon-morning
```

```
no founder rules apply here (0 in scope, none matched; mode=lexical)
Gates: 0 of 0 applicable returned. A result limit cannot hide one.
Soft rules: 0 shown, none omitted.
```

Nothing matched, and that is the truth rather than a bug. The default matching is **lexical**: it compares the words in your query against the words in the rule. The rule says "when I touch *money*"; the query said "VAT" and "order total". No shared word, no match. Say it in words that overlap:

```
python3 tools/bm_learn.py apply \
  --query "change how money is added up in the order total" --session sess-mon-morning
```

```
RELEVANT FOUNDER RULES (mode=lexical)
```

```
4873a92d rank=1
Scope: project:tealeaf      State: approved
When : when I touch money in this project
Do   : keep every amount in integer pence
Why  : a float rounded a customer invoice wrong last month
Match: terms ['money'], relevance 0.143
```

```
recorded 1 application(s), 0 already recorded for this task
retrieval run e63adbfb recorded (1 eligible, 1 returned, limit 5)
```

Two practical lessons, and this is the sort of thing a marketing page would hide. First, **write triggers in the words you will actually use**: say "money" and "rounding" and "price", not a formal category name. Second, the tool told you which mode answered and which terms matched, so you can debug a miss instead of guessing. An optional stemmed index exists behind an environment variable, and every retrieval prints the mode it really used, so no output ever claims a smarter match than it performed.

The line `Gates: 0 of 0 applicable returned. A result limit cannot hide one` is there because of a real defect: a result limit could once drop a blocking rule off the end of the list. Now gates are counted separately from the limit.

Stage 5: outcome

Retrieval is not the end. Say what you did with the rule:

```
python3 tools/bm_learn.py disposition 9efa5994 followed
```

```
application 9efa5994: followed (outcome pending)
not_decidable: followed, but no outcome has been recorded against it yet
```

And when you look at the aggregate:

```
capture metrics (descriptive counts, not accuracy)
  candidates by status: {'approved': 1, 'pending': 2}
  approved rules: 1
```

There is no labelled review set, so none of these is a precision or a recall number, and none of them should be read as one.

Read that last paragraph again. The tool has counts and refuses to dress them up as accuracy. If you ever want to know whether a system is being straight with you, look at what it says about its own metrics.

TRY THIS NOW

In your throwaway project from chapter three, capture a rule that is secretly two rules and watch it warn you:

```
BM=~/.claude/skills/brothermode
python3 $BM/tools/bm_learn.py capture \
  --trigger "when writing tests" \
  --action "use plain asserts and always name the test after the behaviour" \
  --because "clever helpers made a failure unreadable" --scope project
```

Then split it into two captures. You have just done the interpretation step by hand, which is the step every "it learns your preferences" product does silently.

CHAPTER FIVE

How to correct it, and where the correction goes

You will correct it constantly. That is normal and is the input the whole system runs on. What matters is that a correction has somewhere to go.

The three lifetimes of a correction

When you say "no, not like that", you mean one of three things, and it helps to know which:

YOU MEAN	LIFETIME	WHAT TO DO
fix this one thing, here, now	this session	nothing. Just say it. Do not capture it.
from now on, in this project	standing, project scope	capture, approve with scope <code>project</code>
from now on, in all my work	standing, global	capture, approve with scope <code>global</code> , and think twice

The most common mistake is turning row one into row three. A one-off frustration promoted to a global law will fire on unrelated work for months.

What happens to a correction, mechanically

Two paths in. The first you saw in chapter four: you capture it deliberately.

The second is automatic, and it is worth understanding because it is deliberately weak. At the end of a session, a hook scans your short messages against a pattern of correction-shaped phrases ("no, that", "i said", "from now on", "never do") and appends up to five *candidates* per session to a file. They are candidates, not verdicts. A regular expression cannot tell a correction from a quotation, so nothing is imported until you say so:

```
python3 tools/bm_learn.py inbox
```

That prints the queue and ends with a line that tells you nothing has been approved and how to import them. Nothing in that path can create a rule, because the receipt in chapter four stands in the way.

Rules are versioned, not overwritten

A rule you change keeps its history. Ask a rule to explain itself:

```
python3 tools/bm_learn.py why 4873a92d
```

```
rule 4873a92d4a2a4ee6b1d4726b8d2e272e
  4873a92d [project:tealeaf, approved] v1
  When: when I touch money in this project
  Do : keep every amount in integer pence
  Why : a float rounded a customer invoice wrong last month

versions:
  v1 created founder approval (receipt b4440dc0): founder said yes in the
    Monday session;...
evidence (1):
  support founder_approval receipt b4440dc0: founder said yes in the Monday
    session; the refun...
```

That is the audit trail: version, who approved it, which receipt, and the reason recorded at the time. Three verbs move a rule without destroying it:

deprecate

stop retrieving it, keep the history. The usual choice.

supersede

this newer rule replaces that older one, and the link between them is recorded.

reject

for a candidate that never became a rule. The reason is kept, so the same suggestion does not keep coming back.

Rejecting is not deleting, and the difference shows up immediately:

```
python3 tools/bm_learn.py reject fd93a141 --because "it joined two rules into one"
```

```
rejected fd93a141. The reason is kept, so the same suggestion does not keep
coming back.
```

There is a **forget** verb for genuinely removing a rule from future retrieval, and it makes you say so twice:

```
forget removes a rule from every future retrieval. Re-run with --yes if that is
what you want.
```

When a correction contradicts one you already approved

Rules that fight each other are the failure mode of every "learned preferences" system, because the newest one silently wins and you never find out. There are two commands for this, **conflicts** to list them and **resolve-conflict** to settle one, and the resolution is recorded rather than inferred from timestamps.

Grading a rule that hurt you

The last verb closes the loop back to chapter four's fifth question. If work went wrong *because* a rule was followed, say so:

```
python3 tools/bm_learn.py outcome <record-id> --kind rework \  
  --note "what happened"
```

The two kinds are `rework` (it had to be redone) and `escaped_defect` (it shipped and was wrong). This is the only input that can make a rule look bad in the record, which is exactly why it needs to be easy to type on a bad afternoon.

TRY THIS NOW

Approve a rule, then walk it through its whole life without deleting anything:

```
BM=~/.claude/skills/brothermode  
python3 $BM/tools/bm_learn.py capture --trigger "when naming a file" \  
  --action "use lowercase with dashes" --because "mixed case broke a lookup" \  
  --scope project  
# note the id it prints, then:  
python3 $BM/tools/bm_learn.py grant-approval <id> --answer "yes, lowercase dashes"  
python3 $BM/tools/bm_learn.py approve <id> --receipt <token> --ref "trying the flow"  
python3 $BM/tools/bm_learn.py why <rule-id>
```

Now try `approve` a second time with the same token, and watch it refuse. A receipt that could be replayed would be decoration.

CHAPTER SIX

Gates and deep dives: reviewing a machine's decision

A **gate** is a moment where a decision needs a human before it becomes standing behaviour. Chapter four's approval receipt is the mechanism. This chapter is about the harder half: giving the human enough to decide with.

Asking Priya "should we recompute totals in a migration or at read time?" in a chat message is not a review. She needs the code, the callers, the tests, the blast radius and the way back. That is what a **deep dive pack** is.

Nothing is generated until you ask

Packs cost time to produce and go stale if nobody reads them, so none exists until someone asks for one. First, the cheap question: what are the stakes, and where would the pack live?

```
python3 tools/bm_packs.py stakes 33275f67
```

```
stakes: candidate 33275f67 in scope project:tealeaf; 1 file(s) in its declared  
change set  
deep dive on demand: python3 tools/bm_packs.py pack 33275f67  
it would be written to Documentation/30-decisions/D-5-recompute-stored-order-totals-in-a-migra.md
```

That one line is what goes in front of the human at the gate. If the stakes are small, nobody spends the review.

The change set is discovered, not asserted. "1 file(s) in its declared change set" came from the files the work record claimed as its fence. Link the correction to the work it came from with `--record <lifecycle>` when you capture it, and the change set arrives for free. Skip that and the pack will honestly say zero files, which is less useful and is not a lie.

The pack

```
python3 tools/bm_packs.py pack 33275f67 --cite src/pricing.py:7-12
```

```
wrote Documentation/30-decisions/D-5-recompute-stored-order-totals-in-a-migra.md  
8 section(s), 1 citation(s) re-read from disk, 1 human block(s) preserved
```

Eight sections, always in the same order, so a reviewer learns the shape once:

#	SECTION	THE QUESTION IT ANSWERS
1	The decision	in one paragraph of plain language, and what happens if it is wrong
2	Options	each with its trade-off, plus the recommendation and its reason
3	The code	live excerpts with path and line range, re-read at generation time
4	Dependency map	every caller, test and document that touches those lines, found by searching
5	Risks	what breaks, how far it reaches, and the rollback command
6	What the store already knows	related approved rules, prior decisions here, prior recorded failures
7	Diagram	the affected path
8	Review	slots for a named engineer's verdict, notes and residual concerns

Sections three and four are the ones that make it a review rather than a summary. Here is the real output:

3. The code

`src/pricing.py` lines 7 to 12

<!-- bm-cite: path=src/pricing.py lines=7-12 sha256=d0fbc24f9f28... anchor="" -->

```
```python
7
8 def order_total(lines):
9 total = 0
10 for unit, qty in lines:
11 total += line_total(unit, qty)
12 return total
```
```

4. Dependency map

Discovered by scanning 3 text file(s) under the project root for these terms:

- `pricing.py` (the cited file, by name)
- `pricing` (the cited module, as an import or an attribute)
- `order\_total` (a symbol defined in the cited lines)

****Callers and other code: 0.****

- none found

****Tests: 1.****

- `tests/test\_pricing.py:6` matched `pricing, order\_total`

****Documents: 1.****

- `STATE.md:7` matched `pricing.py, pricing`

Note "Discovered by scanning 3 text file(s) ... for these terms", with the terms listed. The map is a search you can re-run, not a claim you have to trust. And when no test references the cited lines, section five says so in the plainest possible words: "A change here is unverified by construction, which is itself the finding."

The property that makes a citation worth reading

Stale quoted code is worse than no quoted code, because a reviewer approves the wrong thing with full confidence. So the pack re-reads every citation when it regenerates, and refuses if the lines have moved.

Watch it. Insert two lines at the top of the cited file, then regenerate:

```
refused (citation-changed): src/pricing.py lines 7-12 still start with the cited
anchor but their content changed since this pack was written. Read the new text
and re-cite with --recite src/pricing.py:7-12@return unit_pence * quantity to
record that you did.
```

There are two refusals here, not one, and which one you see depends on the first line you cited. Line 7 of this file is blank, which is why the recorded anchor above is empty, so nothing can be checked against it and the content hash is what catches the edit: `citation-changed`, at the range you cited. Cite a first line that has text on it and the same edit produces `citation-moved` instead, naming the line the anchor slid to and printing the shifted range. Either way you get a `--recite` command you can paste, and neither one can be satisfied by accident. Do as it says:

```
python3 tools/bm_packs.py pack 33275f67 \  
  --recite "src/pricing.py:7-12@return unit_pence * quantity"
```

```
wrote Documentation/30-decisions/D-5-recompute-stored-order-totals-in-a-migra.md  
  8 section(s), 1 citation(s) re-read from disk, 1 human block(s) preserved
```

An objection with teeth

This is the best part of the chapter, so read the four commands slowly.

Priya reads the pack and finds a real problem. She records it as a note, of kind `alert`, at severity `critical`, anchored to the decision:

```
python3 tools/bm_learn.py note --kind alert --severity critical \  
  --anchor candidate:33275f67 --author "Priya" --author-kind human \  
  --body "the migration has no dry run mode, so a wrong VAT rate rewrites live invoices with no  
  way back"
```

```
note 291c1755 recorded  
  291c1755 alert critical open candidate:33275f67... by Priya  
  this REFUSES an approval anchored here until it is resolved, or until the founder  
  overrides it AT THAT GATE with a recorded reason. An override at one gate does  
  not clear it for the next.
```

Now the founder, who has not read the alert, tries to approve:

```
python3 tools/bm_learn.py grant-approval 33275f67 --answer "yes do it"
```

```
refused (unresolved-critical-alert): a critical alert is unresolved and stands in  
  front of this approval: Priya wrote 'the migration has no dry run mode, so a  
  wrong VAT rate rewrites live invoices with no way back' about candidate 33275f67  
  (note 291c1755). Resolve it, or re-run with an explicit override reason.  
  1 unresolved critical alert(s) match this approval.
```

An engineer's objection stopped a founder's approval. Not by convention, and not by anybody remembering to check: by the command refusing. It names the alert, the author and the anchor, so the founder cannot claim not to have seen it.

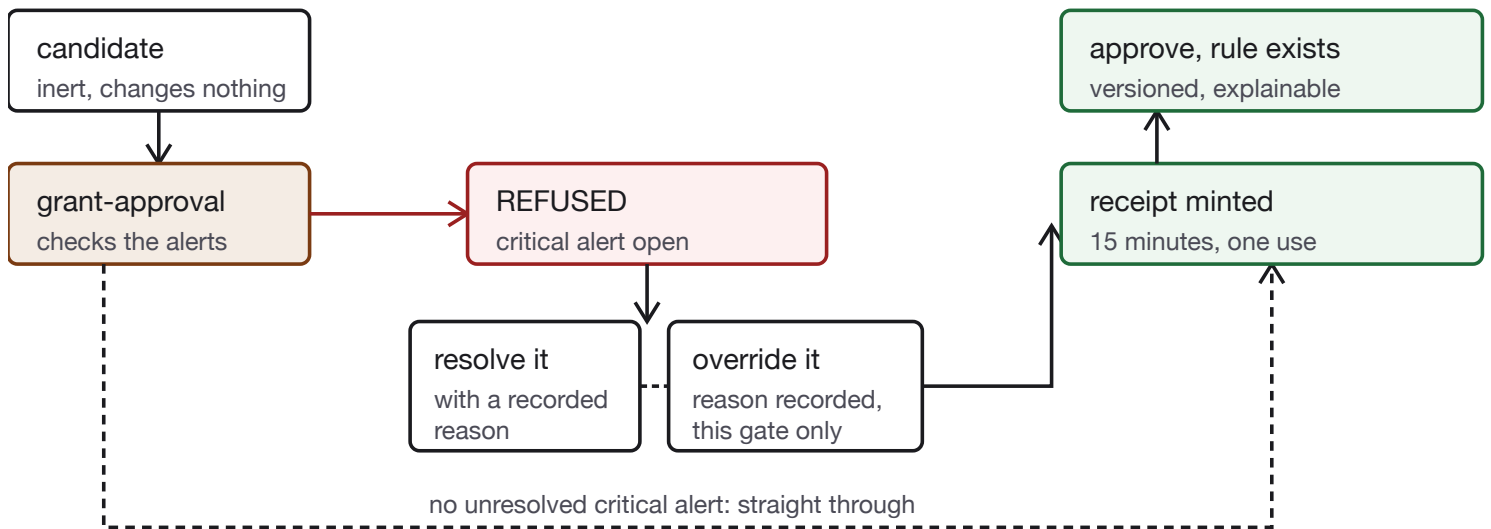
Two ways forward, and both are recorded. Override it, and the override reason is stored while the alert stays visible as overridden rather than being deleted, and it does not carry over to the next gate. Or fix the actual problem:

```
python3 tools/bm_learn.py resolve-note 291c1755 \  
  --because "the migration now has --dry-run, and Priya re-reviewed it on Tuesday"
```

```
note 291c1755 resolved  
291c1755 alert critical resolved candidate:33275f67... by Priya
```

Now the gate opens, and only now:

```
approval receipt 8d23a0c0 for candidate 33275f67  
expires 2026-07-30T02:08:08Z (900 seconds), one candidate, one use
```



Every route to a rule passes through a human answer, and an unresolved critical alert anchored to this candidate closes the route drawn here.

An alert guards the gates it is anchored to, which is narrower than every gate, and you should know the difference before you rely on it. The refusal fires when the unresolved critical alert is anchored to that candidate, to the work record the candidate came from, or to any file in the change set the approval would touch. Nothing anchors an objection to the WORDS of a correction, only to a candidate, a record or a file, so capture the same correction a second time and you get a new candidate with nothing anchored to it, and that gate opens. The objection does not chase the wording around. (Once a rule exists, the duplicate detector does compare wording and refuses the repeat, but that is a different guard and it is not watching for anybody's objection.) Anchor the alert where the risk actually lives instead:

```
python3 tools/bm_learn.py note --kind alert --severity critical \
  --anchor file:src/pricing.py --author "Priya" --author-kind human \
  --body "no dry run mode on anything that rewrites this file"
```

```
refused (unresolved-critical-alert): a critical alert is unresolved and stands in
front of this approval: Priya wrote 'no dry run mode on anything that rewrites
this file' about file src/pricing.py (note c82261b2). Resolve it, or re-run with
an explicit override reason. 1 unresolved critical alert(s) match this approval.
```

That second refusal caught a candidate captured later, with different wording, purely because of the file it would change. A file anchor and a record anchor outlive the candidate. A candidate anchor does not, so it is the weakest of the three and the one to avoid when the objection is really about the code.

Recording the verdict where it counts

A verdict written only in the file is a verdict the system cannot report, so the pack tells you the command:

```
python3 tools/bm_packs.py review 33275f67 --by "Priya" --verdict approve \
  --notes "read the migration, dry run mode exists and is tested" \
  --residual "no test for a rate change mid month"
```

```
review 1624113c recorded against candidate 33275f67
regenerate the pack to list it: python3 tools/bm_packs.py pack 33275f67
```

`--residual` is the field most review tools do not have: what still worries you after you approved. Six months later that sentence is the most valuable line in the pack.

And your own words survive

Every pack ends with a pair of markers. Anything you type between them is carried through every regeneration, byte for byte, including through the refusal above. The author of this book verified that by planting a sentence,

forcing the citation refusal, and checking the sentence was still there:

```
<!-- bm-human:begin -->
Priya: the migration needs a dry run mode. I will not approve without one.
<!-- bm-human:end -->
```

TRY THIS NOW

Prove the refusal to yourself, because a guard you have not seen fire is a guard you do not believe in. In your throwaway project, with a pack that cites a real file:

```
BM=~/.claude/skills/brothermode
python3 $BM/tools/bm_packs.py pack <candidate> --cite app.py:1-3
printf '\n# a new first line\n' | cat - app.py > tmp && mv tmp app.py
python3 $BM/tools/bm_packs.py pack <candidate>
```

It will refuse and tell you the exact `--recite` command. That is what an honest citation looks like.

CHAPTER SEVEN

The Documentation folder, toured

One command writes a folder a stranger can read. Not a wiki you have to maintain: a folder generated from what the project recorded plus what is in the files right now, that you can regenerate any time and that never eats your own writing.

First, the depth question

A one-file script does not need an architecture document. A regulated, multi-contributor system does. So the tool measures the project and picks a depth, and it shows its work:

```
python3 tools/bm_docs.py tier
```

```
tier 2 (standard), chosen automatic
  tier 2 because 2 recorded gate(s), so decisions have been put to a human
  tier 2 because 5 recorded decisions
signals:
  activity_span: under a week of recorded activity
  contributor_names: ['Priya']
  contributors: 1
  gates: 2
  modules: 1
  open_risk_flags: 0
  recorded_decisions: 5
  test_files: 1
  tracked_files: 3
  tracked_files_source: the git index
  work_records: 1
would emit 12 file(s):
  Documentation/00-START-HERE.md
  Documentation/10-business/BA-SUMMARY.md
  ...
```

| TIER | FITS | EMITS |
|------------|--|---|
| 1 lean | a script, a single-file fix, a spike | start here, the decision index, the handover |
| 2 standard | most projects | tier 1 plus requirements, work breakdown, schedule, architecture, data model, dependencies, runbook |
| 3 full | multi-contributor, long-lived, regulated | tier 2 plus process diagrams, a code map, a whitepaper, optional exports |

One rule about depth is worth knowing: **it goes up on its own and never down on its own.** As the project grows, the folder gets deeper without you asking. It will never quietly delete your architecture page because this week looked quiet. Lowering it is an explicit instruction, and the tool says so out loud when you give it:

```
wrote 4 file(s) under Documentation at tier 1 (lean), chosen explicit --tier
  tier 2 because 2 recorded gate(s), so decisions have been put to a human
  lowered to tier 1 by an explicit --tier 1, below the tier 2 the signals measured,
  which is the only way documentation depth goes down
```

Generate it

```
python3 tools/bm_docs.py generate
```

```
wrote 12 file(s) under Documentation at tier 2 (standard), chosen automatic
  tier 2 because 2 recorded gate(s), so decisions have been put to a human
  tier 2 because 5 recorded decisions
...
0 human block(s) preserved verbatim
2 note(s), 0 line anchor(s) checked against the files on disk
narrative: 0 block(s) reused against unchanged facts, 3 regenerated
tier 2 recorded in .brothermode/docs-tier.json, which is what stops a later
  automatic run from lowering it
critical path: f1a2f8b4 (1 recorded day(s))
```

Every line of that report is a promise being kept in public. Human blocks preserved. Note anchors checked against the real files. Narrative blocks reused when the underlying facts have not changed, so a regeneration does not churn prose and leave you reviewing a meaningless diff.

Documentation/

| | |
|------------------|--|
| 00-START-HERE.md | the tier, the signals, the reading order |
| 10-business/ | BA-SUMMARY, REQUIREMENTS, WBS, SCHEDULE |
| 20-technical/ | ARCHITECTURE, DATA-MODEL, DEPENDENCIES
(tier 3 adds PROCESS-DIAGRAMS, CODE-MAP) |
| 30-decisions/ | INDEX plus one deep dive pack per decision |
| 40-handover/ | HANDOVER, RUNBOOK |
| 90-generated/ | facts.json, what every page above came from |

The numbers are the reading order. A stranger does not have to guess where to start.

Numbered folders, so the order a newcomer should read them in is not a matter of taste.

Regeneration does not churn

A generator that rewrites every byte on every run is unusable in a team, because every review opens on a diff that means nothing. The author ran the generator twice and compared checksums:

```
find Documentation -type f | sort | xargs shasum > h1
python3 tools/bm_docs.py generate
find Documentation -type f | sort | xargs shasum > h2
diff h1 h2 && echo "IDENTICAL"
```

```
IDENTICAL
```

The page that matters most

Of the twelve files, one is the reason to bother: `40-handover/HANDOVER.md`. It is written for a person who has never spoken to an assistant about this project and is not going to. Real output:


```
# Handover
```

```
## 1. What this project is
```

In the words recorded on its own work records, this project exists to: Add VAT to the order total without floating point rounding drift. Mechanically it is 1 module(s) and 1 test file(s). Its one gate is `python3 -m unittest discover`.

```
## 2. Current state, and the command that proves it
```

Run this first, before you believe anything else on this page:

```
python3 -m unittest discover
```

It is the command this project declares. If it does not pass, the state below is the state somebody last recorded and not the state you have.

```
## 3. What is done
```

- nothing is recorded as finished

```
## 4. What is in flight
```

- `**vat-rounding**` (`f1a2f8b4`): Add VAT to the order total without floating point rounding drift
- holds: `src/pricing.py`

```
## 6. What is not started
```

- decision D-1 (`cdca32fd`, pending) is waiting on a human: (no action recorded)

```
## 7. Where the code lives, module by module
```

- `src/pricing.py`, 15 lines: Price a tea order. All money is in whole pence.

```
## 9. The known traps
```

- alert [critical] by Priya, resolved, anchored candidate:33275f67...: the migration has no dry run mode, so a wrong VAT rate rewrites live invoices with no way back

Section two is the sentence that distinguishes this from every status document you have written. It does not say "the project is in good shape". It hands you a command and tells you that if it fails, everything below is a claim rather than a state.

Section nine is the other one to notice. That trap is Priya's alert, still on the page after it was resolved, because a resolved objection is history a newcomer needs, not clutter to sweep up.

Diagrams, and where an empty field is honest

The schedule page carries a gantt chart and a critical path, both computed from the recorded work rather than typed:

```
gantt
  dateFormat YYYY-MM-DD
  title Recorded work
  section active
    vat-rounding :f1a2f8b4, 2026-07-30, 1d
```

Dependencies

****Edges: 0.****

– none: no two records claim overlapping files, so every record could have run at any time

That is the house style for an empty field, and it is worth copying in your own work. It does not say "no dependencies". It says zero edges, and it says *why* zero is the right answer here. Where a field is genuinely empty, these pages say the field is empty and name the command that would fill it. Nothing invents a rationale, because a generator that invents a rationale puts words in your mouth.

PDF, Word and Excel, honestly

There is an optional exporter. Nothing imports it; the only way it runs is you running it. Ask what it can do on your machine:

```
python3 tools/bm_docs_export.py report
```

optional export formats on this machine:

pdf NOT available

no writer for pdf is installed here. Tried reportlab, fpdf, weasyprint, none importable. BrotherMode adds no dependency to get it: install one yourself if you want this format, or use the markdown that is already on disk.

docx NOT available

no writer for docx is installed here. Tried docx, none importable.

xlsx available via openpyxl

openpyxl is importable in this interpreter and is used to write the work breakdown and schedule tables as a workbook

markdown plus mermaid is always produced and is already in Documentation.

That is what a missing feature should look like: which format, why not, what it would have been for, and what you can do instead. It never produces a worse file under a name that claims a format.

TRY THIS NOW

Prove the human-block promise on your own machine, because it is the promise you will rely on most:

```
BM=~/.claude/skills/brothermode
python3 $BM/tools/bm_docs.py generate
# open Documentation/00-START-HERE.md, find the two bm-human markers at the bottom,
# and type a sentence between them. Save. Then:
python3 $BM/tools/bm_docs.py generate
grep -A2 "bm-human:begin" Documentation/00-START-HERE.md
```

Your sentence is still there. Now run `bm_docs.py tier` and read the signals: that is the tool telling you what it thinks your project is.

CHAPTER EIGHT

Working with a team

Everything so far worked with one person and one session. A team adds four problems: two workers in the same file, work that has to move between people, work that has to move between machines, and another team in a different repository whose schedule you depend on. This chapter takes them in that order, and is honest about which of the four is solved mechanically and which is not.

Problem one: parallel work in one repository

The rule is **one writer per file**. It is enforced in two places, which is worth understanding because the two catch different mistakes.

At claim time, the store refuses an overlapping claim. Two workers on separate files is fine:

```
python3 tools/bm_store.py claim vat-helper --lifetime ephemeral \
  --objective "write the VAT helper" --files src/vat.py \
  --session "$A" --tier T2 --check "python3 -m unittest discover -s tests -q"

python3 tools/bm_store.py claim vat-tests --lifetime ephemeral \
  --objective "write the VAT tests" --files tests/test_vat.py \
  --session "$B" --tier T2 --check "python3 -m unittest discover -s tests -q"
```

```
claimed 'vat-helper' as lifecycle 6f898731e12c4a88a3626fbabd1e8a7c
(version 1, session bm1-5ead318804d9802408f0b721)
claimed 'vat-tests' as lifecycle b9f9d17c525f481399f80b2e78978d25
(version 1, session bm1-9fcb4ca0eb551f7e4f07c51b)
```

A third worker who wants a file already held is refused, by name, with both paths shown:

```
refused (overlap): claim overlaps active record 'vat-helper'
(lifecycle 6f898731e12c4a88a3626fbabd1e8a7c): 'src/vat.py' vs 'src/vat.py'
```

At write time, the hook refuses the actual edit, which catches the worker who never claimed anything at all. This is the transcript from the throwaway project, and it is the single most useful thing in this chapter:

```
# the owner edits the file it claimed
echo '{"session_id":"sess-mon-morning", ...,
      "tool_input":{"file_path":"src/pricing.py"}}' | python3 tools/bm_fence_hook.py
```

(no output, exit 0: allowed)

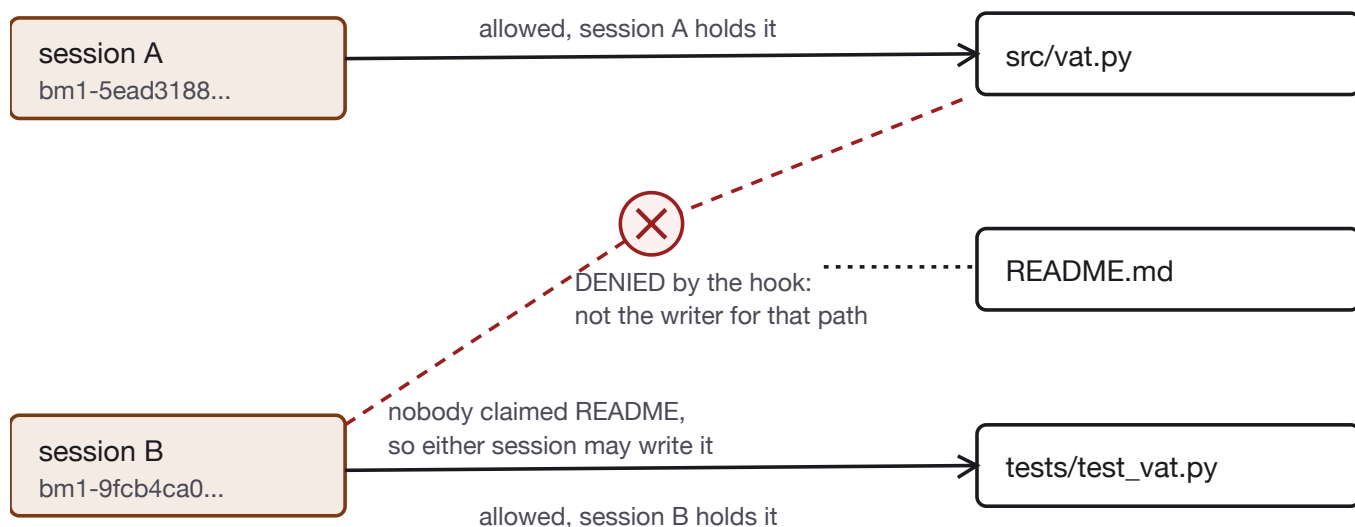
```
# Priya's session edits the same file
echo '{"session_id":"sess-mon-priya", ...,
      "tool_input":{"file_path":"src/pricing.py"}}' | python3 tools/bm_fence_hook.py
```

```
{"hookSpecificOutput": {"permissionDecision": "deny", "permissionDecisionReason":
"BrotherMode fence: src/pricing.py is inside the fence of the active record
vat-rounding (lifecycle f1a2f8b4..., version 1), which is owned by session
bm1-c03e7b279b1cfe4b835074f8. This session is bm1-c1be77e462287fc2f2001a66, so it
is not the writer for that path. Claimed as: src/pricing.py.
One writer per file is structural, not advisory: do not edit across the fence.
Report the needed change to the owner instead. To take the fence over
deliberately, run:
  python3 tools/bm_store.py adopt f1a2f8b4... --version 1
  --session bm1-c1be77e462287fc2f2001a66 --adopt-from-live-session
then re-claim the paths you need with `bm_store.py claim ...`.}}}
```

```
# Priya edits a file nobody claimed
echo '{"session_id":"sess-mon-priya", ..., "tool_input":{"file_path":"README.md"}}' \
  | python3 tools/bm_fence_hook.py
```

(no output, exit 0: allowed)

Three properties in that refusal are worth naming. It says *which* record holds the file and who owns it, so the reader knows who to talk to. It tells you what to do instead, in one sentence. And it prints the exact takeover command, including the version number, so a deliberate takeover is one paste and an accidental one is impossible.



The fence is a set of paths, not a lock on the whole project. What nobody claimed, anybody may write.

Three limits of the fence, stated plainly.

It fails open. A missing store, a corrupt store, a store with no active claims, or a bug in the hook allows the write and prints a line beginning `bm_fence_hook: FAILING OPEN`. The author reproduced this by adopting a record without re-claiming its paths: the store then had no active claims and the very next edit was allowed with that exact warning. Failing closed would brick editing, which is worse, so it is a choice; but it means **you have to read the warning**.

Bash is not gated. `sed -i`, a shell redirect, `tee`, `python -c`: all of these write files without passing any hook. This is policy, published in the project's own documentation.

Adopting is not claiming. Taking a record over transfers the record, not the fence. Re-claim the paths afterwards or you are working unfenced. The refusal message tells you this, in its last line, and it is easy to skim past.

Problem two: handing work to another person

The mechanism is a **parked record with a written handover**. Park carries a heading that is stored in the same transaction as the state change, so there is no window in which the work is parked and the explanation is lost.

```
python3 tools/bm_store.py park 3e2d2f10... --version 1 --session "$ME" \
  --handover "VAT helper is written and tested. The migration is NOT started. Next: add --dry-run
to migrate_vat.py before it touches a live row."
```

```
parked: 'vat-rounding' (lifecycle 3e2d2f10a0e6454ca014b1b611046d5a) is now parked
at version 2
```

Next morning, Priya asks what is waiting:

```
python3 tools/bm_store.py handovers
```

```
handovers: 1 undelivered
- 6004857e7c9b456bac2e15c129a053c0 lifecycle 3e2d2f10a0e6454ca014b1b611046d5a
  2026-07-30T01:55:06Z
  VAT helper is written and tested. The migration is NOT started. Next: add
  --dry-run to migrate_vat.py before it touches a live row.
```

The same text is rendered into `STATE.md`, so anyone opening the project sees it without knowing any commands. She acknowledges it, takes the record, and the fence follows:

```
python3 tools/bm_store.py handover-ack --handover 6004857e...
python3 tools/bm_store.py resume 3e2d2f10... --version 2 --session "$PRIYA" \
  --note "Priya picking up the migration"
```

```
handover-ack: 6004857e... acknowledged; it no longer renders into STATE.md.
active: 'vat-rounding' (lifecycle 3e2d2f10...) is now active at version 3
```

Now the fence has flipped. Priya's write is allowed and the previous owner's is refused, and neither of them had to remember anything:

```
priya: (no output, exit 0: allowed)
monday: {"permissionDecision": "deny", ... "owned by session bm1-ca164dbf73a..."}
```

The one wrinkle worth knowing. `--handover` is real and it works, and it is not listed in the `park` usage line. Run `park` with no arguments and you get `usage: park <lifecycle_uuid> --version N [--session SID] [--note TEXT]`, with no mention of it. Type it anyway. A `--note` is a note to yourself; a `--handover` is a document for the next person, and only the second one shows up in `handovers` and in `STATE.md`.

Problem three: the same person, a new machine or a dead session

This is the same mechanism with a different motive, plus two hooks. Before a compaction (the moment your tool discards old context to make room, which is exactly when work goes missing), a hook snapshots your entire working tree, untracked files included, into a private git reference. It never touches your branch, your index or your working tree, and it never pushes anywhere.

Recovering does not overwrite anything either:

```
python3 tools/bm_autosave.py recover
```

```
bm_autosave: recovered snapshot 81aa7526... into a NEW worktree at:
/var/folders/.../bm-autosave-recover-7ay6aggs
permissions: 0700 (owner-only; best-effort only on platforms, e.g. Windows,
that do not fully enforce a POSIX mode)
Your live working tree at /private/tmp/bmbook/tealeaf was never touched. Inspect
the folder above, copy back what you need, then run
`git -C ... worktree remove ...` when you are done with it.
```

And it warns you about what it captured, which matters if a team member has secrets sitting untracked:

```
bm_autosave: WARNING: this snapshot captures 20 untracked file(s) that no commit
has ever reviewed: Documentation/00-START-HERE.md, ... Anything secret in that
list belongs in .gitignore or in BROTHERMODE_AUTOSAVE_EXCLUDE.
```

The owner-only guarantee is POSIX only. The recovered folder is readable only by you because of a file mode of 0700. Windows governs access differently and the project's own limits page records this failing on both Windows legs of its continuous integration. On Windows, treat a recovered worktree as readable by anyone with an account on the machine.

Problem four: another team, another repository

Here is the honest answer: **this is coordinated, not enforced.** A store is per repository. Two repositories are two stores that know nothing about each other, and the author verified exactly that. In a second repository, the payments team claimed their own file:

```
claimed 'charge-idempotency' as lifecycle 8e52ad87bd864e91a8329c2e06f29987
(version 1, session bm1-5008d7e9bf0215ae3a25c3a1)
```

From the tea repository, that record is invisible. Counting mentions of it in the tea project's own dashboard returns zero. There is no cross-repository lock and the book will not pretend there is one.

What does work, and works well enough to plan against, is that each repository generates a plain text description of its own state that the other team can read:

```
grep -A3 "^## active" ../payments/STATE.md
```

```
## active
- charge-idempotency (8e52ad87bd864e91a8329c2e06f29987, version 1, persistent) [T2]
  owner-session: bm1-5008d7e9bf0215ae3a25c3a1
  objective: make charge idempotent before tealeaf starts sending VAT totals
  files: api/charge.py
```

That is a working cross-team protocol, and it needs no new software:

1. **Each repository commits its Documentation/ folder.** The store is local; the generated folder is the shared artefact. That is why regeneration being byte-identical (chapter seven) matters: a document that churns is a document nobody reviews.
2. **Depend on the other team's decision index, not their chat.** Documentation/30-decisions/INDEX.md is the contract surface: every decision, its state, and the pack that reviews it.

3. **File your objection in their repository, anchored.** A note of kind `alert` at severity `critical` in their store refuses their gate. It is the one cross-team mechanism with real teeth, and it works because you are writing in the repository whose gate you want to stop.
4. **Write the dependency into the objective.** The payments objective above literally says "before tealeaf starts sending VAT totals". Not elegant. Readable by everyone, at the top of a committed file, forever.

A tool you might reach for and should not trust here. `bm_telemetry.py fence-lint` is documented as printing the live fences from a project's `STATE.md` plus any repositories you point it at through the `BROTHERMODE_REGISTRIES` variable, which sounds exactly like cross-repository fence awareness. The author ran it against two repositories with two active fences and got:

```
fence-lint: no live fences found under /private/tmp/bmbook/tealeaf
```

The reason is a format mismatch: it matches an older hand-written fence line format, and the store-generated `STATE.md` does not use it. Use the `grep` above instead. This is reported to the project rather than papered over here.

A team convention that costs nothing and saves afternoons

Claim narrowly and claim late. A fence over `src/` blocks four people; a fence over `src/vat.py` blocks nobody. If you genuinely need a wide fence, use `--lifetime ephemeral` so it is understood to be short, and remember that a fence past its stated lifetime is treated as released rather than blocking the team forever.

TRY THIS NOW

Play both people yourself. It takes three minutes and it is the fastest way to believe the fence:

```
BM=~/.claude/skills/brothermode
cd /tmp/firstloop
A=$(python3 $BM/tools/bm_fence_hook.py session-label --session-id person-a)
B=$(python3 $BM/tools/bm_fence_hook.py session-label --session-id person-b)
python3 $BM/tools/bm_store.py claim mine --lifetime ephemeral \
  --objective "hold app.py" --files app.py --session "$A" --tier T1

printf '{"session_id":"person-b","cwd":"%s","tool_name":"Edit",
"tool_input":{"file_path":"app.py"}}' "$PWD" | python3 $BM/tools/bm_fence_hook.py
```

You will get the deny, with the takeover command in it. Then send the same payload with `person-a` and get silence, which is what allowed looks like.

CHAPTER NINE

What to hand over, and what to keep

A book that says everything is a good fit will lose you at your first bad experience. So this chapter starts with the tasks where BrotherMode is a waste of your time.

Do not use it for these

| TASK | WHY THE CEREMONY LOSES |
|---|---|
| A one-line obvious edit. Fix a typo, bump a version string, change a colour you already decided on. | The claim, the check and the evidence cost more than the edit. There is no decision to review and nothing to remember afterwards. Just make the edit. |
| Anything you are still thinking out loud about. | Capturing a rule from an opinion you will revise in an hour pollutes the retrieval for weeks. Think first, capture second. |
| A throwaway exploration you will delete. | A work record and a fence for code with no future is pure overhead. The exception: if it might teach you something, record the one decision at the end and throw the rest away. |
| Reading, browsing, asking a question. | Nothing is written, so there is nothing to fence, prove or hand over. |
| Work where you are the only person and the file is yours forever. | The fence is coordination machinery. With nobody to coordinate with, its only value is the evidence gate at the end, and you can get that from your test runner. |

There is a command that will tell you this, and it is refreshingly willing to say no. It takes signal flags rather than a description of the task:

```
python3 tools/bm_learn.py should-retrieve
```

```
retrieval advised: no
  no substantial signal fired, so retrieval here would be ceremony rather than care
  nothing was recorded, because a task that did not happen must not appear in the
  outcome data
```

Read the second line. It will not add a "no" to the ledger, because a ledger padded with trivial non-tasks makes every later number meaningless. Now declare the signals that are actually true of your work:

```
python3 tools/bm_learn.py should-retrieve --multi-file --risky --prior-correction
```

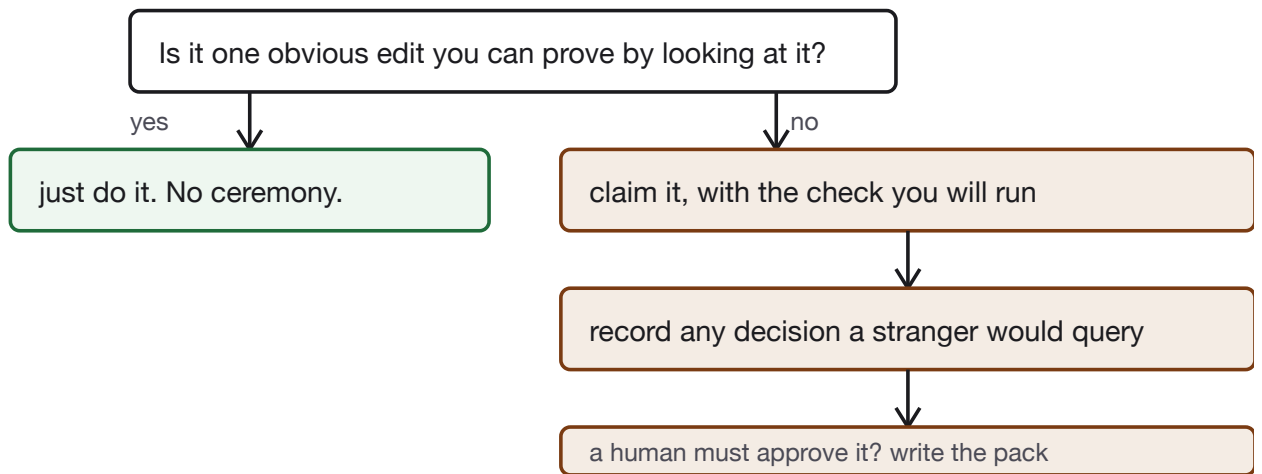
```
retrieval advised: yes
  it changes more than one file
  it is a risky or irreversible operation
  a correction has landed in this area before
```

The available signals are `--multi-file`, `--risky`, `--architecture`, `--communication` and `--prior-correction`. That list is the most compact statement of what this system is for that exists anywhere in the project.

Do use it for these

| TASK | WHICH PART EARNS ITS KEEP |
|--|--|
| Anything touching money, dates, identity or deletion | Rules and gates. These are the areas where a forgotten preference costs a refund. |
| A change spanning more than one file | The fence, and the dependency map in the pack. |
| A decision somebody will ask about later | <code>decide</code> , and a deep dive pack if it needs a human verdict. |
| Work you will not finish today | Checkpoints and the handover. This is the highest return for the least typing in the whole system. |
| Two or more workers at once | The fence, on both sides. |
| Anything you have already had to correct once | Capture it. The second correction is the one that tells you it is a pattern. |
| A migration, a rename, a schema change | Everything. Rollback in the pack, evidence at completion, a critical alert if a reviewer is not satisfied. |

The one-minute version



Most work takes the left branch. That is not a failure of the system; it is what keeps the right branch worth doing.

TRY THIS NOW

Take the last three things you asked an assistant to do. For each, run:

```
BM=~/.claude/skills/brothermode
python3 $BM/tools/bm_learn.py should-retrieve          # nothing declared
python3 $BM/tools/bm_learn.py should-retrieve --multi-file --risky
```

and decide which flags were honestly true. If none of the three earned a yes, you have learned something real: start with the fence and the evidence gate, and leave the rules until you have been bitten twice.

CHAPTER TEN

Other runtimes: what works, what is unverified

Nothing in BrotherMode's engine is specific to any one AI tool. It is standard library Python driven from a shell. But *wiring* is specific, and this chapter exists to keep one distinction from being blurred, because blurring it would be dangerous rather than merely sloppy.

Two different questions

Question 1: HOOK POINTS

Does this tool run an external program at lifecycle events at all?

Answered per runtime from the vendor's own page, with the date it was read.

VERIFIABLE, and verified.

Question 2: BROTHERMODE'S HOOKS

Do BrotherMode's own hook programs work there?

They parse Claude Code's payload format. A same-named event is not a same payload.

UNVERIFIED outside Claude Code.

UNVERIFIED means do not wire it. A fence that fails open while looking installed is worse than no fence.

Every honest cross-runtime claim lives in one of these two columns, and most marketing puts them in the same sentence.

Ask the project itself:

```
python3 tools/bm_runtimes.py list
```

BrotherMode runtime adapters (conventions read 2026-07-29)

| | |
|--------------|---|
| generic | Generic AGENTS.md |
| file: | generic.AGENTS.md |
| hook points: | none found. Fence is advisory here. |
| codex | OpenAI Codex CLI |
| file: | codex.AGENTS.md |
| hook points: | yes (11 events). BrotherMode hook compatibility UNVERIFIED. |
| copilot | GitHub Copilot |
| file: | copilot.copilot-instructions.md |
| hook points: | none found. Fence is advisory here. |
| antigravity | Google Antigravity |
| file: | antigravity.brothermode.md |
| hook points: | yes (5 events). BrotherMode hook compatibility UNVERIFIED. |
| qwen | Qwen Code |
| file: | qwen.QWEN.md |
| hook points: | yes (16 events). BrotherMode hook compatibility UNVERIFIED. |
| iflow | iFlow CLI |
| file: | iflow.IFLOW.md |
| hook points: | none found. Fence is advisory here. |

Store and retrieval commands work in ALL of these: they are ordinary local processes.

What that table actually means for you

| RUNTIME | READS THE LAW FROM | STORE AND PACKS AND DOCS | AUTOMATIC HOOKS |
|-----------------------|---------------------------------|--------------------------|--|
| Claude Code | CLAUDE.md and SKILL.md | yes | yes, the one verified runtime |
| Generic
AGENTS.md | AGENTS.md | yes | no hook points found at all |
| OpenAI Codex
CLI | AGENTS.md | yes | has hook points; payload shape never captured, so unverified |
| GitHub Copilot | .github/copilot-instructions.md | yes | no hook points found at all |
| Google
Antigravity | .agents/rules/brothermode.md | yes | has hook points; unverified |
| Qwen Code | QWEN.md | yes | closest event list to Claude Code's; still unverified |
| iFlow CLI | IFLOW.md | yes | no hook points found at all |

So the practical shape of using another tool is this. **Everything you do by typing a command works.** The store, the fence claims and their refusals, the rules, the packs, the documentation folder: all of it is an ordinary local program reading standard input and writing standard output. It needs no verification per runtime, and the project says so.

Everything automatic is Claude Code only, today. The five hooks parse Claude Code's payload format. Nobody has captured a real payload from any other runtime, which is why every non-Claude answer in that column says UNVERIFIED rather than "probably fine". Qwen Code names `SessionStart` and `SessionEnd`, the two events the telemetry actually targets, which makes it the most promising target and still not a verified one.

Why the distinction is a safety matter, not pedantry

Consider wiring the fence hook into a runtime whose payload does not carry a session identifier where BrotherMode expects one. The hook cannot tell who is writing, so it fails open, and failing open is silent by design. You would have a project with a `PreToolUse` entry in its configuration, a team who believe one writer per file is enforced, and no enforcement whatsoever. That is strictly worse than knowing you have no fence, because a team that knows will be careful.

Two further honest notes the project makes about itself. In a runtime with no hook points, the fence is **advisory**: BrotherMode can tell the tool the law and the tool can run the store commands, but no gate stands in front of a write, so one writer per file is a coordination ledger rather than a boundary. And there is no automatic installation

of adapter files, because `AGENTS.md` at a repository root usually already has content in it, and a generator that overwrote it would be a data loss bug wearing a convenience feature's clothes.

TRY THIS NOW

Generate the adapter file for whatever tool you actually use, and read it. It goes into a staging directory and prints where each file belongs; it installs nothing:

```
BM=~/.claude/skills/brothermode
python3 $BM/tools/bm_runtimes.py list
python3 $BM/tools/bm_runtimes.py emit --runtime generic
```

Then open the generated file and check one thing: does it promise your tool anything the table above says is unverified? It should not. If it does, that is a bug worth reporting.

CHAPTER ELEVEN

When it goes wrong

Sorted by how likely you are to hit it. Every message quoted here was produced by the author on purpose.

"refused (stale-identity)"

```
refused (stale-identity): expected version 3 in a state that allows -> active;  
found version 3 state 'adopted'
```

What happened. Every mutating command takes `--version N`, the version you last saw. If the record has moved since, the command refuses rather than overwriting whatever happened in between. In the case above, the version was right but the state was wrong: the record had been adopted, and adopted does not move straight back to active.

What to do. Look at the record, then use what you see:

```
python3 tools/bm_store.py dashboard
```

This is not an error to work around. It is the thing standing between you and a lost afternoon on Wednesday.

"FAILING OPEN, the write is allowed and the fence was NOT checked"

```
bm_fence_hook: FAILING OPEN, the write is allowed and the fence was NOT checked.  
Reason: the store at ../brothermode/store.sqlite3 holds no active claims, so  
there is no fence to enforce
```

What happened. The most common cause is the one the author hit while writing chapter eight: a record was adopted but its paths were never re-claimed, so no active claim existed and there was nothing to enforce.

What to do. Re-claim the paths, then confirm they are there:

```
python3 tools/bm_store.py claim <name> --files <paths> --session "$ME"  
python3 tools/bm_store.py dashboard
```

Never treat this line as noise. It is the exact wording that separates "the fence allowed it" from "there was no fence".

The rule you approved never fires

What happened. Almost always word overlap. Matching is lexical by default, as chapter four showed: a rule triggered on "money" does not match a query about "VAT".

What to do. Run the retrieval and read the diagnostics it prints for free, including the mode and the matched terms:

```
python3 tools/bm_learn.py lookup --query "<the words you would really use>"
```

`lookup` reads and records nothing, so it is the safe verb for debugging. Then rewrite the trigger in the vocabulary you actually type. There is an optional stemmed index behind `BROTHERMODE_FTS5=1`, and `index-status` reports whether it is really answering, but fixing the words is usually the better fix.

The hooks are wired and nothing happens

Three causes, in order of likelihood. The settings file is not valid JSON, in which case the whole hooks block is ignored and nothing says so. Or the tool has not restarted, and hook configuration is read at startup. Or the session was too short: the telemetry has a deliberate activity floor and does not record a trivial session, so the ledger cannot be padded.

```
python3 -m json.tool ~/.claude/settings.json
python3 ~/.claude/skills/brothermode/scripts/doctor.py
```

A pack refuses to regenerate

```
refused (citation-changed): src/pricing.py lines 7-12 still start with the cited
  anchor but their content changed since this pack was written. Read the new text
  and re-cite with --recite src/pricing.py:7-12@return unit_pence * quantity to
  record that you did.
```

Working as designed. Read the new code, then pass the `--recite` argument it handed you. Note what that does: it records that a human looked. Passing `--cite` again will not clear it, and that is deliberate.

Health check, when something feels off but nothing has errored

```
python3 tools/bm_store.py verify
```

```
verify: healthy, 0 problem(s)
```

Getting your work back

```
python3 tools/bm_autosave.py recover
```

If nothing was ever saved it says exactly that, rather than pretending:

```
bm_autosave: no autosave found for /private/tmp/bmbook/tealeaf (the latest pointer
  is empty and no snapshot ref is newer than the last clean check).
  No snapshot refs exist for this worktree at all.
```

Two things that will annoy you and are not broken

A pile of `STATE.md.bak-...` files. Every render backs up the previous file first. The author's throwaway project accumulated seven of them in about fifteen minutes of normal use. They are safe to delete and the tool will not delete them for you, because a tool that decides on its own which of your files are disposable is a tool you cannot leave alone with a project. Add `STATE.md.bak-*` to your `.gitignore`.

`--help` is not always `--help`. Several commands print their usage only when you give them no arguments at all, and treat a flag as data otherwise. Running `bm_store.py claim --help` creates a work record actually named `--help`, which the author did while researching this book and had to clean up. Run the bare verb (`bm_store.py claim`) to see usage. For the learning tool, an unrecognised flag is properly refused and it lists the flags it does know, which is the behaviour you want:

```
bm_learn: unrecognized flag --query (recognized: --architecture, --communication,
  --json, --multi-file, --prior-correction, --risky)
```

The limits worth knowing before you rely on this

These are from the project's own `docs/KNOWN-LIMITS.md`, not from a critic:

- **It has never been run through a real working day.** Everything rests on tests, adversarial execution and simulated lifecycles.
- **Continuous integration has failed on Windows.** On the tagged release commit the Windows job exited 1, and two other legs were cancelled rather than passing, so their result is unknown rather than green.
- **Recovered work is owner-only on POSIX only.** Not on Windows.
- **No independent audit has been run.** Every finding in the project was found by the same family of model that wrote the code.
- **Treat any test count in the repository as a claim to re-verify.** A mutation audit found fifteen tests described as calibrated that were exercising a local copy of old code and could never have failed. They were deleted and the honest count reported. That paragraph is in the project's own limits page, which tells you most of what you need to know about whether to trust it.

TRY THIS NOW

Cause a refusal deliberately, so the first one you see is not in real work:

```
BM=~/.claude/skills/brothermode
cd /tmp/firstloop
python3 $BM/tools/bm_store.py dashboard          # note a lifecycle and its version
python3 $BM/tools/bm_store.py complete <uuid> --version 99
```

You get a stale-identity refusal naming the version it actually found. Now run it with the right version and no `--evidence` , and get the missing-evidence refusal. Two refusals, two minutes, and the system stops being mysterious.

CHAPTER TWELVE

Glossary

Alphabetical. Every entry describes what the thing is at commit `471efdc` , not what it might become.

agent

A model working on a task with tools, in its own conversation. When you ask an assistant to spawn helpers, each helper is an agent. In BrotherMode each one is expected to hold a fence and to declare a tier, so its spend can be judged against a declared intent.

alert

A kind of note carrying a severity. An unresolved alert at severity `critical` , anchored to a decision or to a file that decision would change, refuses the approval. Chapter six shows the refusal.

anchor

Where a note is attached: a file and optionally a line, a decision, a gate, or a work record. Notes render at their anchors in the generated pages. A note whose anchored line has moved is reported rather than silently dropped.

candidate

A captured correction that has not been approved. Completely inert: it changes nothing at all until a human approves it, and it can be rejected with a reason so the same suggestion stops coming back.

change set

The files an approval would affect, discovered from the store rather than asserted: the claimed paths of the work record the correction came from, plus the scope key when the scope is a single file.

check

The command that proves a piece of work. Declared at claim time with `--check` , before you know how the work will go, because afterwards you will accept anything as proof.

claim

Registering a unit of work and the exact set of files it may write. The second half is the fence.

compaction

The moment your tool discards older conversation to make room for new. Useful and necessary, and the moment work most often disappears, which is why a hook snapshots your files and your next intent just before it happens.

deep dive pack

A generated markdown document for one decision, in eight fixed sections, complete enough for an engineer to review the choice without reading a transcript. Generated on demand and never before.

evidence

The pasted result of running the check. Required, in a non-empty form, to move work to complete. There is no flag to skip it.

fail open

When something goes wrong, allow the action rather than block it. The fence hook fails open on purpose: a broken hook that blocked every edit would be worse than an unenforced fence. It always says so out loud, and that line is the one you must never ignore.

fence

The set of files one worker may write. Enforced twice: the store refuses an overlapping claim, and a hook refuses the actual edit. It does not cover writes made through shell commands.

gate

A moment where a decision needs a human before becoming standing behaviour. Also used for a check severe enough to block work landing, as opposed to a soft check that is only scored.

handover

Two related things. A heading written when work is parked, stored in the same transaction so the explanation cannot be lost, and listed until someone acknowledges it. And a generated page, `40-handover/HANDOVER.md`, written for a human who will never talk to an assistant.

hook

A small program your tool runs automatically at a fixed moment: session start, session end, before an edit, before a compaction. The point is that it does not depend on the model remembering. BrotherMode wires five. Verified in Claude Code only, per chapter ten.

human block

Text between a pair of markers in a generated file. Preserved byte for byte through every regeneration, including through a refused one. It is the mechanism that makes a generated document safe to write in.

lifecycle

The permanent identity of one unit of work, a long hex string that nothing else ever reuses. Two things that look like the same task are two lifecycles.

loop

One unit of work with a beginning, a middle and a provable end. Called a loop because the evidence closing one is what the next one reads.

note

An authored row attached to an anchor, of kind `insight`, `alert`, `question`, `review`, `todo` or `risk`, carrying who wrote it and when. Resolved notes stay, marked resolved. No generator deletes one.

park

Putting work down without finishing it. With `--handover` it also writes the document for whoever picks it up.

receipt

A single-use, fifteen-minute token minted when someone records what a human actually answered about a specific rule. No receipt, no rule. It dies if the rule text changes, and there is no override. It proves an answer was given about this exact thing; it does not prove who gave it, and nothing in the product claims otherwise.

rule

An approved standing preference with a trigger, an action, a reason and a scope. Versioned, explainable through `why`, and retrievable. Retired by deprecating or superseding it, never by erasing its history.

scope

How wide a rule reaches: `global`, `project`, `domain`, `artifact`, `relationship` or `tool`. The most common mistake in the whole system is making a project preference global.

session label

The public name for one working session, derived from a secret token stored in your project. Get it from `bm_fence_hook.py session-label`; never type one by hand, because an invented label mints a token nobody can reproduce and locks you out of your own records.

store

The one database file, per project, under `.brothermode/`. One program is the only writer of it. Everything else asks that program.

tier (documentation)

How deep the generated `Documentation/` folder goes: 1 lean, 2 standard, 3 full. Chosen from measured signals, with the reason printed. Rises on its own, never falls on its own.

tier (effort)

How much work you are budgeting: T1 one session, T2 a few helpers, T3 a fleet with divided scopes. Declared on the claim so that spend is judged against an intent rather than a feeling.

unverified

Used in this project as a technical term, not a hedge. It means nobody has captured the evidence, so acting as though the capability exists would be guessing at a safety boundary. Every non-Claude entry in the hooks column of chapter ten is unverified, and unverified means do not wire it.

vault

A plain folder of markdown and text files holding your session logs, the machine-written ledger and optionally a model of how you like to work. One per person. Nothing in it talks to a server. Movable with the `BROTHERMODE_VAULT` variable.

version (of a record)

A counter every mutating command must state. If the record moved since you last looked, the command refuses instead of overwriting. This is the mechanism behind most of the refusals in this book, and it is the reason two workers cannot silently clobber each other's state.

TRY THIS NOW, ONE LAST TIME

Pick the three entries above you would not have been able to explain before reading this book. Then run the command that makes each one concrete: `dashboard` for the store and versions, `bm_docs.py tier` for tiers, `bm_learn.py why <rule>` for rules and receipts. A definition you have seen a command produce is a definition you keep.

About this book

Written against BrotherMode 2.0.0-rc.4 at commit `471efdc`. Every command shown was executed by the author in a throwaway git repository built for the purpose, and every block labelled as output is pasted from that run. Session labels, lifecycle identifiers and receipt tokens will differ on your machine; that is correct.

Four things the author found while writing it, which are reported to the project rather than smoothed over here: `bm_store.py claim --help` creates a work record named `--help`; `--handover` is absent from the `park` usage line although it is the whole person-to-person handover mechanism; `bm_telemetry.py fence-lint` finds no fences in a store-generated `STATE.md`; and every render leaves another `STATE.md.bak-...` file behind.

There are no productivity figures in this book because there is no honest source for one. When BrotherMode has been through real working days and the ledger has real rows in it, that will be a different book, and it will cite the command that produced each number.